

Genre API Documentation

1. Create Genre

Creates a new genre in the video streaming application.

Endpoint

POST

```
/api/genre/
```

Full URL

```
https://api/api/genre/
```

Replace `https://api` with your actual backend/base API URL.

Reference

[Genre Create API Documentation](#)

2. Request

HTTP Method

```
POST
```

Content-Type

```
Content-Type: application/json
```

Request Body

The request body must contain the following fields:

Field	Type	Required	Description
<code>name</code>	string	Yes	Name of the genre

Field	Type	Required	Description
slug	string	Yes	Unique URL-friendly identifier for the genre
description	string	Yes	Description of the genre

Request Body Example

```
{
  "name": "Action",
  "slug": "action",
  "description": "Movies and series containing action-oriented stories and sequences."
}
```

3. Successful Response

HTTP Status

201 Created

A successful request returns the newly created genre.

Response Fields

Field	Type	Required	Description
id	integer	Yes	Unique ID generated for the genre
name	string	Yes	Name of the genre
slug	string	Yes	URL-friendly genre identifier
description	string	Yes	Description of the genre

Response Example

```
{
  "id": 3365,
  "name": "Action",
  "slug": "action",
  "description": "Movies and series containing action-oriented stories and sequences."
}
```

```
sequences."  
}
```

4. Request → Response Flow

```
Client  
|  
| POST /api/genre/  
|  
| JSON Body  
| {  
|   "name": "Action",  
|   "slug": "action",  
|   "description": "..."  
| }  
|  
v  
Backend API  
|  
| Validate request  
| Create Genre  
| Generate ID  
|  
v  
Database  
|  
| Save Genre  
|  
v  
Backend API  
|  
| 201 Created  
|  
v  
Client
```

5. Python Example

```
import requests  
  
url = "https://api/api/genre/"
```

```

payload = {
    "name": "Action",
    "slug": "action",
    "description": "Movies and series containing action-oriented stories and
sequences."
}

headers = {
    "Content-Type": "application/json"
}

response = requests.post(
    url,
    json=payload,
    headers=headers
)

print(response.status_code)
print(response.json())

```

Expected Output

```

{
  "id": 3365,
  "name": "Action",
  "slug": "action",
  "description": "Movies and series containing action-oriented stories and
sequences."
}

```

6. JavaScript Example

```

const url = "https://api/api/genre/";

const options = {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    name: "Action",
    slug: "action",

```

```
        description: "Movies and series containing action-oriented stories and sequences."
    })
};

try {
    const response = await fetch(url, options);
    const data = await response.json();

    console.log(response.status);
    console.log(data);
} catch (error) {
    console.error(error);
}
```

7. cURL Example

```
curl -X POST "https://api/api/genre/"
-H "Content-Type: application/json"
-d '{
    "name": "Action",
    "slug": "action",
    "description": "Movies and series containing action-oriented stories and sequences."
}'
```

8. Postman Configuration

Method

POST

URL

https://api/api/genre/

Headers

Key	Value
Content-Type	application/json

Body

Select:

Body → raw → JSON

Then enter:

```
{
  "name": "Action",
  "slug": "action",
  "description": "Movies and series containing action-oriented stories and
sequences."
}
```

Expected Result

Status: 201 Created

```
{
  "id": 3365,
  "name": "Action",
  "slug": "action",
  "description": "Movies and series containing action-oriented stories and
sequences."
}
```

9. Field Validation

The API expects all three request fields.

name

Type: string
Required: Yes

Example:

```
"name": "Action"
```

slug

Type: string
Required: Yes

Example:

```
"slug": "action"
```

The slug should normally be lowercase and URL-friendly.

Good:

```
action  
science-fiction  
romance  
crime-thriller
```

Avoid:

```
Action Genre!!!
```

description

Type: string
Required: Yes

Example:

```
"description": "Movies and series based on action stories."
```

10. Example Genres

Action

```
{
  "name": "Action",
  "slug": "action",
  "description": "Fast-paced movies and series involving action and adventure."
}
```

Comedy

```
{
  "name": "Comedy",
  "slug": "comedy",
  "description": "Movies and series created primarily for entertainment and humor."
}
```

Thriller

```
{
  "name": "Thriller",
  "slug": "thriller",
  "description": "Suspenseful movies and series designed to create excitement and tension."
}
```

11. API Summary

Property	Value
API Name	Create Genre
Method	POST
Endpoint	/api/genre/

Property	Value
Request Format	JSON
Response Format	JSON
Success Status	201 Created
Authentication	Not specified in the provided API definition
Request Fields	name, slug, description
Response Fields	id, name, slug, description

12. Minimal API Definition

POST /api/genre/

Request

```
{
  "name": "Action",
  "slug": "action",
  "description": "Action movies and series."
}
```

Response — 201 Created

```
{
  "id": 3365,
  "name": "Action",
  "slug": "action",
  "description": "Action movies and series."
}
```

13. API Testing Checklist

- [] Verify the endpoint accepts POST requests.
- [] Verify a genre is created with valid name, slug, and description.
- [] Verify the response status is 201 Created.
- [] Verify the response contains a generated integer id.
- [] Verify the response contains the submitted name.

- [] Verify the response contains the submitted `slug`.
- [] Verify the response contains the submitted `description`.
- [] Verify missing `name` is handled correctly.
- [] Verify missing `slug` is handled correctly.
- [] Verify missing `description` is handled correctly.
- [] Verify invalid data types are rejected.
- [] Verify duplicate `slug` behavior according to the backend validation rules.
- [] Verify invalid JSON is rejected.
- [] Verify the `Content-Type` requirement.
- [] Verify authentication/authorization if enabled by the backend.

14. Notes

The examples above use `https://api/api/genre/` as the base URL because that is the URL shown in the supplied API definition. For your actual application, replace `https://api` with the deployed backend URL or your local development server.

The provided API definition does not specify error-response schemas, authentication requirements, pagination, or duplicate-slug behavior. Those should be added to the documentation once they are defined by the backend.